第一章 金融科技

如果说第 1 章讨论的是金融的「经济学」——现金流的定价与风险的补偿，那么本章讨论的是金融的「工程学」——数据如何被采集与存储、信用如何被量化、支付如何被清算、模型如何被治理。金融科技（FinTech）常被描述为技术对金融的颠覆，但对量化从业者更准确的定位是：**技术改变了金融的生产函数**。它降低了信息获取成本、提高了风险计量的分辨率、压缩了交易与清算的时间尺度，但没有改变风险与收益必须匹配这一约束。

核心要点 1.1

FinTech 的价值来源可以归结为三类成本的下降：**搜寻成本**（把资金供需双方匹配起来，如 P2P 与智能投顾）、**验证成本**（确认交易对手的身份与信用，如 eKYC 与另类数据评分）、**执行成本**（完成交易与清算，如实时支付与分布式账本）。任何金融科技商业模式的可持续性，都可以用「它把哪一类成本降低了多少、降低了谁的」来检验。

1.1 金融科技的定义与演进

1.1.1 四个阶段

1. **FinTech 1.0（1866–1967）**：以电报、跨洋电缆与清算所为核心，金融信息第一次跨越地理边界高速传递，跨国支付与外汇市场得以形成。

2. **FinTech 2.0（1967–2008）**：ATM、信用卡网络、SWIFT 报文与电子交易所出现，金融业务从纸面迁移到电子系统。此阶段的关键是**流程标准化**。

3. **FinTech 3.0（2008–至今）**：移动支付、P2P 借贷、机器人投顾、区块链与云计算。此阶段的关键是**渠道去中介化**：终端用户可以直接触达金融产品的提供者。

4. **FinTech 4.0（正在形成）**：嵌入式金融（金融服务嵌入非金融场景）、开放银行、以生成式人工智能为核心的新型交互层。此阶段的关键是**决策自动化**。

补充说明　分期的用意

上述分期是行业惯例而非严格史实，争议点在于「3.0 是否已结束」。分期的价值在于提示每一阶段的核心约束：1.0 的约束是通信带宽，2.0 的约束是流程与凭证，3.0 的约束是信任与获客成本，4.0 的约束则是**模型可靠性与合规责任**。约束在哪里，监管与技术的焦点就在哪里。

1.1.2 技术栈的分层

金融科技的技术栈可以分成四层，各层的失败模式完全不同，见表 表 1.1。理解分层很重要，因为一次生产事故的责任归属往往取决于它发生在哪一层。

表 1.1 金融科技技术栈分层与失败模式

层次	典型组件	核心职责	主要失败模式
数据层	数据仓库、数据湖、Kafka、对象存储	采集、存储、版本化与血缘追踪	时点错配（point-in-time 缺失）、字段口径不一致、上游断流
计算层	Spark、Flink、Python 与 SQL 引擎、特征平台	特征加工、模型训练与批量评分	训练与推理特征不一致、回填数据泄露未来信息
模型层	评分卡、GBDT、深度学习、大语言模型	风险计量与决策输出	分布漂移、过拟合、不可解释导致合规拒绝
应用层	开放银行 API、支付网关、智能投顾前端	交互、授权与执行	认证失效、接口抖动、限额与风控规则冲突

1.2 数据基础设施

1.2.1 数据仓库、数据湖与湖仓一体

数据仓库（Data Warehouse）要求写入前先定义模式（schema-on-write），数据经过清洗与建模后以星型或雪花模型组织，适合结构化报表与指标口径统一。**数据湖**（Data Lake）先存储原始数据再按需解释（schema-on-read），适合非结构化数据与探索性分析，但容易退化为缺乏治理的「数据沼泽」。**湖仓一体**（Lakehouse）在对象存储之上叠加事务表格式（如 Delta Lake、Iceberg、Hudi），同时提供 ACID 事务与开放格式读取能力。

⚠ 风险警示　数据湖的三种典型事故

一是**口径漂移**：同一指标在报表、风控与营销三个系统中定义不同，导致同一客户在不同系统里风险等级相反。二是**时点错配**：用今天已修订的财务数据回测三年前的策略，相当于让策略读到未来信息，回测收益会被系统性高估。三是**血缘缺失**：上游字段含义变更无人知晓，模型输入悄然改变，表现衰减却被归因为「市场变了」。解决这三类问题的工程手段是统一的指标口径层、point-in-time 快照表与字段级血缘追踪。

1.2.2 批处理、流处理与批流一体

批处理（Batch）按固定周期处理有界数据集，吞吐高、延迟以小时或天计；流处理（Stream）处理无界数据流，延迟可到毫秒级，但需要处理乱序、重复与恰好一次语义。两种常见的融合架构是：

- **Lambda 架构**：批处理层提供准确的历史视图，速度层提供近似的实时视图，查询时合并。代价是同一套逻辑要维护两份实现，容易出现口径不一致。
- **Kappa 架构**：只用流处理，历史数据通过重放消息队列重建。逻辑单一，但对消息保留期与重放成本要求高。

对风控场景而言，延迟与准确性之间存在明确的经济权衡：实时反欺诈需要在毫秒内决定是否放行，因此只能使用预先计算的轻量特征；而贷后监控与模型迭代可以使用小时级或天级的完整数据。把两类需求混在一条链路里，通常导致两边都做不好。

实务应用　事件时间与处理时间

在流式风控中必须区分**事件时间**（交易实际发生的时刻）与**处理时间**（系统收到并计算该事件的时刻）。若以处理时间为窗口，一笔因网络延迟在次日到达的交易会被计入次日的统计，日终汇总与实时监控因此出现差异。工程上的做法是以事件时间为准并使用水位线（watermark）容忍有限迟到，同时明确「窗口关闭后是否允许修正」的策略并写入文档。

1.3 信用评分卡

信用评分是量化金融在金融科技领域最成熟、也最具经济价值的应用。一个完整的评分卡开发流程包含七个步骤，其中最容易被忽视的是前两步的数据准备与分箱。

1.3.1 从数据到 WOE 与 IV

设目标变量 $y=1$ 表示违约（坏客户）、$y=0$ 表示正常（好客户），$p=\mathbb{P}(y=1\mid \mathbf{x})$ 为违约概率。逻辑回归模型假设违约的对数几率是特征的线性函数：

$$\ln\left(\frac{p}{1-p}\right)=\beta_0+\beta_1 x_1+\beta_2 x_2+\cdots+\beta_k x_k \tag{1.1}$$

其中 $\mathbf{x}=(x_1,\dots,x_k)$ 为特征向量（连续变量通常先分箱并转为 WOE 编码），β_j 为待估系数。由于 式 (1.1) 的右侧可以取任意实数而左侧被限制在 $(-\infty,\infty)$，因此不需要对 p 施加额外约束，这是逻辑回归在评分卡中被长期采用的技术原因之一。反解 式 (1.1) 得

$$p=\frac{1}{1+\exp\!\big(-(\beta_0+\boldsymbol{\beta}^{\top}\mathbf{x})\big)} \tag{1.2}$$

证据权重（Weight of Evidence，WOE）把每个分箱的违约信息压缩为一个数：

$$\mathrm{WOE}_i=\ln\left(\frac{\text{该箱中好客户占比}}{\text{该箱中坏客户占比}}\right)=\ln\left(\frac{n_i^{\mathrm{good}}/N^{\mathrm{good}}}{n_i^{\mathrm{bad}}/N^{\mathrm{bad}}}\right) \tag{1.3}$$

其中 n_i^{good} 与 n_i^{bad} 分别为第 i 箱中好、坏客户数量，N^{good} 与 N^{bad} 为全样本的好、坏客户总数。WOE 为正表示该箱的好客户相对更集中（风险较低），WOE 为负表示风险

较高。WOE 之所以有用，是因为它把非线性关系单调化：在 式 (1.1) 中用 WOE 作为输入，等价于用各箱的对数几率差直接建模，从而使逻辑回归能够捕捉单调但非线性的关系。

信息量（Information Value，IV）度量某个变量对好坏区分的贡献：

$$\mathrm{IV} = \sum_{i=1}^{m} \left(\frac{n_i^{\mathrm{good}}}{N^{\mathrm{good}}} - \frac{n_i^{\mathrm{bad}}}{N^{\mathrm{bad}}} \right) \mathrm{WOE}_i \tag{1.4}$$

其中 m 为分箱数。式 (1.4) 的形式是相对熵（KL 散度）的一种近似，可以理解为「知道该变量后，关于好坏的判断不确定性的减少量」。经验判据为：$\mathrm{IV} < 0.02$ 预测力很弱，0.02 至 0.1 为弱，0.1 至 0.3 为中等，0.3 至 0.5 为强，超过 0.5 时应当警惕——如此之高的 IV 在真实业务数据中罕见，通常意味着该变量泄露了结果信息（例如用了催收记录去预测违约）。

1.3.2 评分刻度：从几率的对数到分数

模型输出的是概率，业务需要的是分数。二者之间用一个线性映射连接：

$$\mathrm{score} = A - B \ln(\mathrm{odds}), \qquad \mathrm{odds} = \frac{1-p}{p} \tag{1.5}$$

其中 p 为违约概率，odds 为好坏比（好客户与坏客户之比），A 为基准分数，B 为刻度因子。两个参数由两项业务约定唯一确定：基准好坏比 O_0 对应的基准分数 S_0，以及好坏比翻倍所需的分数增量 PDO（Points to Double the Odds）。由 式 (1.5) 在两点的差值可得

$$B = \frac{\mathrm{PDO}}{\ln 2}, \qquad A = S_0 + B \ln O_0 \tag{1.6}$$

式 (1.6) 是评分卡部署中最常被写错的公式，把 $\ln 2$ 写成 $\ln 10$ 会让所有分数的间距错约 3.3 倍。代入检验：若设 $S_0 = 600$ 分对应 $O_0 = 50:1$，$\mathrm{PDO} = 20$，则 $B = 20/0.69315 = 28.8539$，$A = 600 + 28.8539 \times 3.91202 = 712.87$。当 $\mathrm{odds} = 20:1$ 时，$\mathrm{score} = 712.87 - 28.8539 \times 2.99573 = 626.4$ 分；当 $\mathrm{odds} = 100:1$ 时，$\mathrm{score} = 712.87 - 28.8539 \times 4.60517 = 580.0$ 分。好坏比从 $20:1$ 升到 $100:1$（扩大 5 倍，即倍数 $\ln 5/\ln 2 = 2.32$ 次翻倍），分数上升 $2.32 \times 20 = 46.4$ 分，与表中结果一致。

例 1.1 由逻辑回归系数直接写出评分卡

假设模型含两个 WOE 编码变量，估计得 $\ln(p/(1-p)) = -2.20 - 0.85\,w_1 - 1.30\,w_2$，其中 w_1, w_2 为两变量的 WOE 值。注意好客户对应低违约率，而 WOE 为正表示好客户集中，因此系数为负是符合预期的。由 $\mathrm{odds} = (1-p)/p = \exp(-(\beta_0 + \beta_1 w_1 + \beta_2 w_2))$，代入 式 (1.5) 得 $\mathrm{score} = A - B\ln(\mathrm{odds}) = A + B(\beta_0 + \beta_1 w_1 + \beta_2 w_2)$。取 $A = 712.87$、$B = 28.8539$，则基准分 $= 712.87 + 28.8539 \times (-2.20) = 649.4$ 分；w_1 每增加一个单位（好客户占比的相对优势提高 e 倍），分数增加 $28.8539 \times (-0.85) = -24.5$ 分——符号之所以为负，是因为本例中 $\beta_1 < 0$ 而 WOE 已在编码方向上调正，实际

部署时必须核对 WOE 的定义方向，否则整张评分卡会反向。更常见的等价写法是把系数与 B 的乘积直接作为每箱的得分（scorecard points），从而把评分卡做成一张「查表相加」的规则表，便于业务与监管审阅。

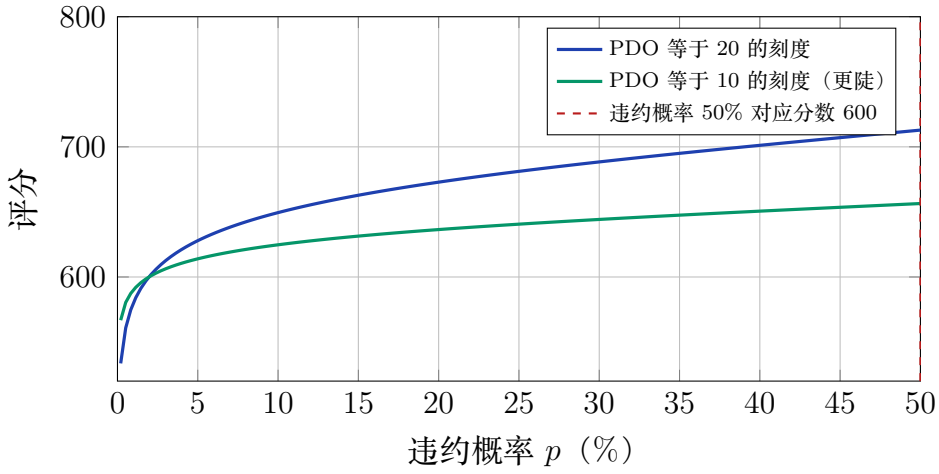


图 1.1 评分刻度映射曲线与 PDO 设定的影响

图 1.1 有两点值得强调。第一，映射曲线在低违约概率区域很陡：p 从 1% 降到 0.5%，好坏比从 99 升到 199，分数上升约 20 分；而在高违约概率区域几乎平坦：p 从 40% 升到 45%，分数只下降约 3 分。这意味着分数的分辨率在不同风险区间完全不同，用统一的分差阈值去区分「高风险名单」会在尾部失效。第二，PDO 越小曲线越陡，同一风险差异对应的分差越大；PDO 的选择应使业务上关心的风险差异落在 20 至 50 分的可解释区间内。

实务应用　分箱的工程约束

分箱（binning）需要同时满足四项约束：每箱样本量不低于总体的 5%（否则 WOE 的估计方差过大）、箱数控制在 5 至 10 个、WOE 随风险单调、且分箱边界可以写成业务语言的阈值（如「征信查询次数 0 次、1 至 2 次、3 至 5 次、6 次以上」）。纯粹由算法优化的分箱若边界不可解释，在监管质询时很难辩护。此外，分箱必须在训练集上确定并冻结，验证集与生产数据使用同一套边界，否则会引入信息泄露。

1.3.3 预期损失与风险调整收益

信用风险的第一性公式是预期损失：

$$EL = PD \times LGD \times EAD \tag{1.7}$$

其中 PD 为违约概率（通常取一年期口径），LGD 为违约损失率（$= 1 -$ 回收率），EAD 为违约时的风险敞口。三个参数的量纲与观测难度并不相同：PD 是最容易从评分卡估计的量，LGD 依赖抵押品处置与清偿顺序，EAD 在循环授信下还取决于违约前的额度使用率（通常用信用转换系数 CCF 建模）。

例 1.2 一笔贷款的预期损失与经济利润

某银行给一家中型企业发放 1 年期贷款，金额 1000 万元（EAD），贷款利率 7.5%，资金成本 3.0%，非预期损失的资本占用按贷款的 8% 计提，股东要求的资本回报率 12%。评分卡给出的 1 年期 $PD = 2.0\%$，历史同类敞口的回收率为 40%，故 $LGD = 60\%$。则

$$
\begin{aligned}
EL &= 0.02 \times 0.60 \times 1000 = 12.0 \text{ 万元}, \\
\text{利息收入} &= 1000 \times 7.5\% = 75.0 \text{ 万元}, \\
\text{资金成本} &= 1000 \times 3.0\% = 30.0 \text{ 万元}, \\
\text{经济利润} &= 75.0 - 30.0 - 12.0 - 1000 \times 8\% \times 12\% = 23.4 \text{ 万元}.
\end{aligned} \tag{1.8}
$$

其中最后一项 $1000 \times 8\% \times 12\% = 9.6$ 万元是资本占用成本。若该笔贷款的定价降到 6.0%，则经济利润变为 75.0 被替换为 60.0，结果为 8.4 万元，仅剩原来的三分之一。这解释了为什么小微企业贷款利率在表面上看「很高」（年化 8% 至 15%），而银行的实际利润仍然微薄：在 $PD = 2\%$、$LGD = 60\%$ 的假设下，仅预期损失就吞掉 1.2% 的利差，再加上资金成本与资本成本，能够留给风险的价差空间非常有限。

在 式 (1.7) 的基础上定义风险调整后的资本收益率：

$$
RAROC = \frac{\text{风险调整后的收益}}{\text{经济资本}} = \frac{\text{收入} - \text{成本} - EL}{\text{经济资本}} \tag{1.9}
$$

其中分母的经济资本对应非预期损失（Unexpected Loss），即在给定置信水平下损失超过 EL 的部分。式 (1.9) 的意义在于把风险显式地放进分母，从而可以比较不同风险等级的业务的资本效率：一笔 $PD = 0.5\%$ 的贷款与一笔 $PD = 6\%$ 的贷款，只有在资本占用与预期损失都被扣除之后，才能判断哪一笔真正为股东创造价值。

表 1.2 巴塞尔框架下的信用风险资本计量方法对比

方法	风险权重来源	数据要求	适用机构与局限
权重法（SA）	监管给定的外部评级映射表	仅需外部评级与敞口分类	中小银行易于实施；对低风险客户资本占用偏高，缺乏风险敏感性
内部评级法基础版（F-IRB）	机构自估 PD，LGD 与 EAD 用监管值	需通过监管验收的 PD 估计与评级体系	资本节约明显；但 PD 估计的顺周期性会把风险放大
内部评级法高级版（A-IRB）	PD、LGD、EAD 全部自估	需长期违约与回收数据、抵押品估值体系	风险敏感性最强；对数据治理与模型验证要求最高

表 1.2 的三种方法不是纯粹的精度递增关系，而是「资本敏感度」与「数据与治理成本」之间的权衡。选择更精细的方法意味着监管资本更贴近真实风险，但也意味着在数据质

量与模型治理上必须支付长期成本。内部评级法的另一个副作用是 **顺周期性**：经济下行时 PD 上升、抵押品价值下降导致 LGD 上升，资本要求随之提高，银行被迫收缩信贷，从而放大下行压力。这正是巴塞尔协议 III 引入逆周期资本缓冲与杠杆率底线的原因。

式 (1.7) 的结构可以用一张图直观拆开。把损失分布的横轴视为损失金额，预期损失是分布的均值，非预期损失是给定置信水平下的分位数与均值之差，而极端损失落在分位点之外，需要按压力情景单独准备资本。

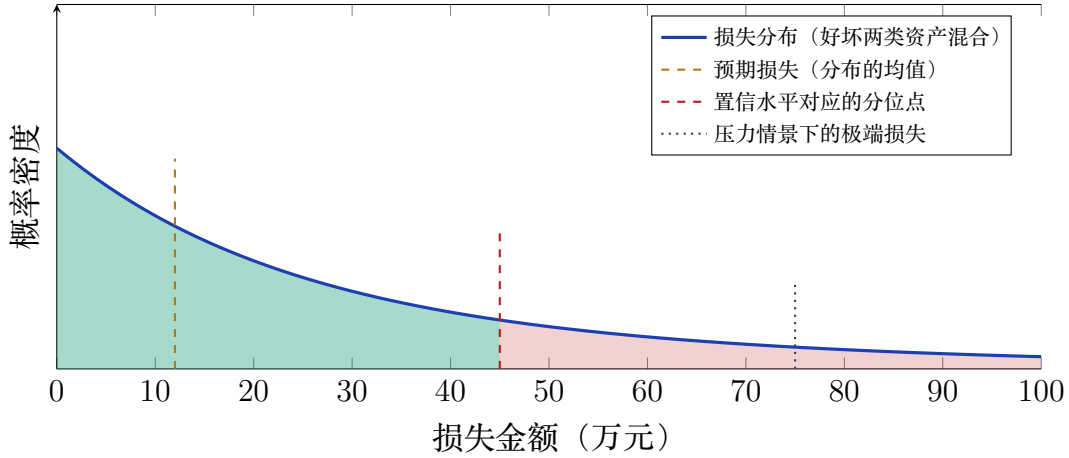


图 1.2　预期损失、非预期损失与极端损失在损失分布上的位置

图 1.2 中绿色区域对应预期损失，红色区域对应超出预期损失的尾部，其面积等于非预期损失发生的总概率。这张图解释了为什么银行同时需要 **拨备**（覆盖预期损失，计入损益）与**资本**（覆盖非预期损失，吸收冲击）：两者覆盖的是分布的不同部分，用拨备去吸收尾部损失会迅速耗尽，而用资本去覆盖预期损失则是低效的资本占用。

⚠ 风险警示　EL 模型的失效条件

第一，EL 是**期望**，不是上限。以 EL 作为损失预算而不保留非预期损失的资本，在 PD 低估时会被尾部损失击穿。第二，PD 通常来自历史样本，若样本期没有经历完整信用周期，估计值会被系统性低估，这一点在经济扩张期尤其危险。第三，三个参数并非独立：PD 与 LGD 在危机中同时上升（同一宏观因子驱动），把二者相乘时隐含的独立性假设会低估真实损失。第四，敞口口径必须与 PD 的期限一致，用一年期 PD 去乘多年期敞口会得到没有意义的数字。

1.3.4　评分模型的监控与压力测试

模型投产只是开始。上线后必须持续监控三类指标：**区分度**（如 KS 统计量、基尼系数、AUC 的月度变化）、**稳定性**（特征分布与分数分布的漂移）、以及**校准度**（预测违约率与实际违约率的一致性，常用二项检验与 Hosmer–Lemeshow 检验）。分布漂移最常用的度量是群体稳定性指数（Population Stability Index，PSI）：

$$\mathrm{PSI} = \sum_{i=1}^{m} (a_i - e_i) \ln\!\left(\frac{a_i}{e_i}\right) \tag{1.10}$$

其中 a_i 为当期第 i 箱的样本占比，e_i 为建模期（基准期）第 i 箱的占比，m 为箱数。式 (1.10) 与 式 (1.4) 结构相同，都是相对熵，只是把「好坏的区分」换成了「时间上的分布变化」。经验阈值为：小于 0.1 视为稳定，0.1 至 0.25 需要关注并做归因，超过 0.25 通常触发模型重训或降级使用。

例 1.3 PSI 的数值计算

某特征分为四箱，基准期占比为 $e = (0.40, 0.30, 0.20, 0.10)$，当期占比为 $a = (0.30, 0.30, 0.24, 0.16)$。逐箱计算 $(a_i - e_i)\ln(a_i/e_i)$：第一箱 $(-0.10)\ln 0.75 = 0.02877$；第二箱 $0 \times \ln 1 = 0$；第三箱 $0.04 \times \ln 1.20 = 0.00729$；第四箱 $0.06 \times \ln 1.60 = 0.02821$。合计 PSI $= 0.0643$，小于 0.1，判定为稳定。但若把当期占比改为 $a = (0.20, 0.25, 0.28, 0.27)$，同样的四箱计算得 0.1155、0.0163、0.0419、0.1326，合计 PSI $= 0.3063$，超过 0.25，触发重训评估。注意 PSI 对尾箱尤其敏感：第四箱占比从 10% 升到 27% 贡献了总 PSI 的 43%，这提示监控报告必须同时给出逐箱贡献，否则无法定位漂移来源。

压力测试与 PSI 监控是互补的：PSI 回答「已经发生了什么变化」，压力测试回答「如果发生某种宏观冲击会怎样」。标准做法是把宏观变量（GDP 增速、失业率、房价指数）与 PD、LGD 建立卫星模型，设定轻度、中度、重度三档情景，计算资本充足率在情景下的路径，并对最脆弱的组合制定处置预案。

1.4 支付与清算基础设施

1.4.1 RTGS 与 DNS

支付系统的清算机制分为两类。**实时全额结算**（RTGS）逐笔全额、实时完成资金划转，消除了结算延迟带来的对手方风险，但要求参与者保持充足的日间流动性；**延迟净额结算**（DNS）在结算周期内轧差后再划转净额，流动性效率高，但在轧差完成前存在敞口——这是「赫斯塔特风险」（Herstatt risk）的来源。现代大型支付系统通常采用「RTGS 处理大额、DNS 处理零售」的混合模式。

表 1.3　主要支付清算机制与标准对比

机制或标准	适用范围	关键特征	主要风险与约束
RTGS	大额跨行支付、证券交收	逐笔全额、实时、央行货币结算	日间流动性压力大，需排队管理与透支便利
DNS	零售批量支付、卡组织清算	周期轧差、净额划转、成本低	结算前存在对手方敞口，需会员信用限额
ISO 20022	跨境与国内报文	结构化数据字段，可承载汇款用途与合规信息	迁移期新旧报文并存，字段映射易丢信息
实时支付系统	零售即时转账	全年无休、秒级到账、限额管理	不可撤销，欺诈与误操作损失难以追回

表 1.3 中的 ISO 20022 值得单独说明：它把传统的定长报文字段替换为可扩展的 XML

结构，使「谁付给谁、为什么付」这类信息能够随款项一起传递，从而支持制裁名单筛查、反洗钱监测与自动对账。其代价是报文体积膨胀与新旧系统的映射成本，跨境迁移期通常需要报文转换网关做双向翻译，而字段语义的丢失是迁移项目中最常见的故障源。

1.4.2 实时支付的运营含义

实时支付把「到账」的等待时间从数小时压缩到数秒，这带来三个结构性变化。第一，**流动性管理**的粒度从日变为秒，参与机构需要在央行账户保持更高的日间余额，或依赖流动性节约机制（LSM）做双边与多边轧差。第二，**欺诈窗口**被压缩，但一旦放行便不可撤销，因此风控必须前移到预授权阶段，模型延迟预算通常在百毫秒量级。第三，**对账逻辑**必须改变：批处理时代的 T+1 对账不再适用，需要基于唯一端到端标识的实时状态机来跟踪每笔支付的生命周期。

补充说明　TPS 数字的口径

行业内常引用「峰值每秒数十万笔」的交易处理能力。比较这类数字时必须核对四点：统计的是单系统还是全链路（含银行核心与清算所）、峰值窗口是秒级还是一分钟均值、是否包含重复计数（如扫码支付中商户收款与用户付款各计一次）、以及是否在营销活动等极端场景下达到。口径不同，同一系统的 TPS 差异可以达到一个数量级，这也是容量规划中必须自建压测基准的原因。

1.5 开放银行与数据合规

1.5.1 开放银行的技术与商业逻辑

开放银行（Open Banking）通过标准化的应用程序接口（API）把账户信息与支付发起能力开放给持牌第三方。其核心不是技术，而是**数据主体对自身数据的控制权**：客户授权第三方读取其在银行的交易记录，或直接发起支付。主要制度包括欧盟的支付服务指令第二版（PSD2）、英国开放银行标准、以及美国的金融数据交换标准（FDX）。

对量化从业者而言，开放银行带来的直接价值是更完整的客户现金流数据，可用于更精细的偿付能力评估；风险则是数据链条变长后新增的攻击面，以及第三方倒闭时的数据处置问题。

1.5.2 合规框架：授权、最小化与跨境

- **欧盟 GDPR**：以「合法基础」为前提处理个人数据，要求目的限制、数据最小化、存储期限限制、数据主体权利（访问、更正、删除、可携带），并对自动化决策（含画像）设置限制与解释要求。
- **欧盟 PSD2**：在支付领域落实账户访问与支付发起两类接口，要求强客户认证（SCA），并规定第三方只能在客户明示授权范围内访问数据。
- **中国《个人信息保护法》**：要求处理个人信息具备合法性基础与单独同意（对敏感个

人信息）、目的明确与最小必要、境内存储与跨境提供的前置条件（安全评估、认证或标准合同）。

⚠ 风险警示 合规的三条红线

其一，**超范围使用**：客户授权读取账户信息用于信贷审批，却被用于营销或转售给关联方，属于典型违规。其二，**无限期留存**：数据保留期限未定义，或删除请求无法在系统层面真正执行（例如备份与特征库中仍有副本）。其三，**跨境与再委托**：把数据传给境外云服务商或其子处理者时未做合规评估。这三类问题在技术上都表现为「数据流图与授权范围不一致」，因此合规审计的第一步永远是画出完整的数据血缘图，而不是检查某一份合同。

1.6 模型风险治理

1.6.1 机器学习模型的特殊风险

传统评分卡由少量变量与单调约束构成，其行为可以被业务人员理解并在一页纸上说明；机器学习模型（尤其是梯度提升树与深度网络）用数百上千个特征拟合复杂的交互关系，精度更高，但带来四类新风险：**过拟合**（样本内表现远优于样本外）、**不可解释**（无法向监管与客户说明拒绝理由）、**脆弱性**（对输入分布的微小偏移敏感，如对抗样本）、以及**可复现性**（依赖库版本、随机种子与数据快照，缺一不可）。

1.6.2 SR 11-7 与三道防线

美国联邦储备委员会 2011 年发布的 SR 11-7 指引给出了模型风险管理的通用框架，其核心是把模型风险治理拆成三个相互独立的角色，见表 表 1.4。

表 1.4 模型风险治理的三道防线与关键动作

防线	承担主体	关键动作	失效表现
第一道：开发与使用	业务条线与模型开发团队	模型文档、假设清单、开发与验证数据分离、上线前自测	自我验证流于形式，问题在投产后才暴露
第二道：独立验证	独立于开发的模型验证团队	概念合理性、数据质量、代码复现、基准对比、敏感性分析与结果分析	验证报告只做合规性陈述，不做真正的复现与挑战
第三道：内部审计	内部审计部门	评估前两道防线的流程有效性与执行一致性	审计只查文档是否齐全，不追查执行证据

表 1.4 中第二道防线的实质是**复现**：独立团队用同一份数据与同一份代码能否得到同样的结果，用更简单的基准模型（如逻辑回归或单一决策树）能否接近复杂模型的精度。若一个复杂模型相对简单模型的提升不足几个基点，却需要付出数倍的验证与维护成本，正确的结论往往是放弃复杂模型。

实务应用 可解释性的工程实现

在信用决策等受监管场景中，模型解释不是锦上添花而是准入门槛。常用手段包括：使用 SHAP 值给出单笔申请的逐特征贡献（可生成「拒绝理由」），用部分依赖图（PDP）与累积局部效应（ALE）检查特征影响是否单调合理，用单调约束（monotonic constraints）在训练阶段强制「收入越高、违约概率越低」，以及用代理模型（如深度不超过三层的决策树）拟合复杂模型的决策面。需要强调的是：解释工具的稳定性本身需要检验，两次运行的 SHAP 值若差异很大，基于它出具的拒绝理由同样不可靠。

1.6.3 大语言模型在金融中的应用与局限

大语言模型（LLM）在金融领域的现实应用集中在文本密集型环节：财报与公告的结构化抽取、合同条款比对、客服问答、代码与研报辅助生成、以及合规材料初筛。它带来的效率提升是真实的，但其失效模式与传统模型截然不同，必须在部署前明确：

- **幻觉**：模型会生成语法流畅但与事实不符的内容，包括虚构的财务数字与不存在的法条。缓解手段是强制检索增强（RAG）并附引用来源、对数值型输出要求逐字摘录原文位置、以及设置「无法确认时回答不知道」的拒答策略。即使如此，也无法保证零幻觉，因此关键决策必须有人类复核环节。
- **不可复现**：同一提示在不同时间、不同温度参数或不同模型版本下可能给出不同答案。监管场景要求可追溯，因此必须固定模型版本、记录完整提示与参数、并保存输出用于事后审计。
- **合规与保密**：把客户数据或未公开信息发送给外部模型服务商可能构成数据出境或泄密。可行方案是本地化部署或使用具备数据处理协议的私有实例。
- **责任归属**：模型没有法律主体资格，最终责任仍在机构与持牌人。把 LLM 输出直接作为投资建议或信用决策依据，在多数司法辖区都不满足「适当性」与「可解释性」要求。

风险警示 不能交给大语言模型的三类任务

一是**最终数值决策**：授信额度、风险定价、投资建议，这些必须有可审计的确定性模型与人工签字。二是**合规判断**：法条适用与制裁名单认定涉及法律责任，模型输出只能作为线索。三是**实时交易执行**：LLM 的延迟与不确定性不适合毫秒级闭环控制。把这三类任务交给 LLM，本质上是用不可验证的组件承担可验证的责任，属于典型的模型风险失配。

1.7 另类数据与量化平台

1.7.1 另类数据的价值与陷阱

另类数据（Alternative Data）指传统财报与行情之外的数据源：卫星影像、信用卡消费流水、应用下载与使用时长、物流与供应链记录、招聘岗位数量、社交媒体情绪等。评

估另类数据时应当依次检验四个问题：

1. **覆盖偏差**（coverage bias）：样本是否代表整体？信用卡面板通常偏向特定银行与特定人群，用它推断全行业消费增速会产生系统偏差。

2. **幸存者偏差**（survivorship bias）：数据集中是否只包含仍然存续的实体？若已退市的公司被剔除，回测中的成长性会被高估。

3. **时点一致性**（point-in-time）：数据在历史上实际可获得的时刻与当前数据集中的时间戳是否一致？很多数据集在后期做过回溯修订，直接用修订后的历史序列回测等于使用未来信息。

4. **拥挤度**：若同一数据源被大量资金使用，其预测能力会被套利活动迅速侵蚀。评估拥挤度可以观察数据供应商的客户数、以及该信号相关策略的容量与换手率。

例 1.4　覆盖率外推的偏差量级

某信用卡面板仅覆盖全国约 8% 的消费支付，且集中于一线城市与特定发卡行。若该面板显示某月消费同比下降 5%，而分析者直接把它当作全国消费增速，当真实全国消费仅下降 2% 时，误差为 3 个百分点。若策略使用该信号做多空配置并假设行业收入弹性为 1.5，则预期的基本面差被放大到 4.5 个百分点，相当于把信号强度高估了一倍以上。正确的处理方式是把面板数据与官方口径做历史回归，估计缩放系数与偏差项，并在样本外持续校验映射关系是否稳定；一旦映射关系断裂（例如发卡行调整了数据上报逻辑），必须先修复映射再使用信号。

1.7.2　量化研究与回测平台

量化策略的标准生命周期为：数据清洗、因子构建、组合构建、回测、绩效分析、实盘执行。工程上最容易出问题的环节是「研究环境与生产环境不一致」：研究用 pandas 在内存中处理全量数据，生产用流式计算分片增量，同一个因子在两处的实现细节（空值填充、复权方式、停牌处理）不同，导致实盘表现与回测不符。

表 1.5 的选择原则是：研究阶段追求**迭代速度**，生产阶段追求**保真度**。可行的做法是在研究阶段用向量化框架做因子筛选，再用事件驱动框架对候选策略做高保真复算，最后用生产同构的撮合模拟做最终确认，每一个阶段的结论都必须能够被上一阶段的实现所解释。

核心要点 1.2　本章的三条主线

一是**数据的时间属性**：无论数据湖、评分卡还是另类数据，最致命的错误都是时点错配——用未来才知道的信息解释过去，见 式 (1.10) 的监控逻辑、表 1.1 的分层与 小节 1.7.1 的外推偏差。二是**风险参数的乘法结构**：$EL = PD \times LGD \times EAD$ 是信用风险的第一性公式，见 式 (1.7)，其失效主要来自参数相关性与样本期偏短。三是**模型的可验证性**：模型的复杂度必须与可验证性匹配，SR 11-7 的三道防线与表 1.4 的第二道防线都以「独立复现」为核心。

表 1.5 主流回测与数据平台的适用场景对比

工具或平台	语言与生态	优势	局限
Backtrader	Python	事件驱动、支持多品种与多周期、文档完整	单机运行，大数据量下性能受限，缺少因子研究能力
Zipline 系列	Python	与因子研究流程衔接好、内置交易日历与滑点模型	维护活跃度波动，扩展自定义数据源成本较高
VectorBT	Python 与 NumPy	向量化回测速度极快，适合参数扫描与因子筛选	事件顺序与撮合逻辑简化，不适合复杂订单类型
自研低延迟引擎	C++ 或 Rust	完全可控、微秒级延迟、可复现生产撮合逻辑	开发与验证成本高，需要独立的正确性测试体系

Listing 1.1 评分卡流程：分箱、WOE 与 IV、逻辑回归与评分刻度映射

```python
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression

def woe_iv(df, feature, target, bins=5):
    """Bin a numeric feature by quantile and compute WOE/IV per bin."""
    df = df[[feature, target]].dropna().copy()
    df["bin"] = pd.qcut(df[feature], q=bins, duplicates="drop")
    g = df.groupby("bin", observed=True)[target].agg(["count", "sum"])
    g.columns = ["n", "bad"]
    g["good"] = g["n"] - g["bad"]
    g["dist_good"] = g["good"] / g["good"].sum()
    g["dist_bad"] = g["bad"] / g["bad"].sum()
    g["woe"] = np.log(g["dist_good"] / g["dist_bad"])
    g["iv_bin"] = (g["dist_good"] - g["dist_bad"]) * g["woe"]
    return g, float(g["iv_bin"].sum())

def score_map(p, base_score=600.0, base_odds=50.0, pdo=20.0):
    """Map default probability to a scorecard score."""
    b = pdo / np.log(2.0)
    a = base_score + b * np.log(base_odds)
    return a - b * np.log((1.0 - p) / p)

rng = np.random.default_rng(7)
n = 4000
data = pd.DataFrame({
    "income": rng.lognormal(10.5, 0.5, n),
    "util": rng.uniform(0.0, 1.0, n),
})
logit = -1.2 - 0.9 * (data["income"] - 10.5) + 2.0 * (data["util"] - 0.5)
data["target"] = rng.binomial(1, 1.0 / (1.0 + np.exp(-logit)))

```

```
table, iv = woe_iv(data, "util", "target", bins=5)
print(table[["n", "bad", "woe", "iv_bin"]].round(4))
print("IV(util) = %.4f" % iv)

data["woe_util"] = data["util"].map(
    lambda v: table["woe"].iloc[min(int(v * 5), 4)])
model = LogisticRegression().fit(data[["woe_util"]], data["target"])
p_hat = model.predict_proba(data[["woe_util"]])[:, 1]
print("intercept=%.4f  coef=%.4f" % (model.intercept_[0], model.coef_[0,
    0]))
print("score at p=0.02: %.1f" % score_map(0.02))
print("score at p=0.01: %.1f" % score_map(0.01))
```

Listing 1.1 把本节的三组公式串成一条可运行的链路：`woe_iv` 实现 式 (1.3) 与 式 (1.4)，`score_map` 实现 式 (1.5) 与 式 (1.6)，中间的逻辑回归对应 式 (1.1)。注意示例中的映射函数只用于说明刻度换算，真实部署时必须把 WOE 分箱表冻结并存档，并用独立的验证样本重新计算 KS 与分数分布，避免示例代码中的样本内估计被误用。

本章小结

- 金融科技的价值来自搜寻、验证、执行三类成本的下降；技术栈分为数据、计算、模型、应用四层，各层的失败模式不同，见 表 1.1。
- 数据仓库与数据湖的关键分歧是模式定义与治理时机；批流一体的两种实现是 Lambda 与 Kappa，选择取决于口径一致性与重放成本。
- 评分卡流程为分箱、WOE、IV、逻辑回归、刻度映射；WOE 与 IV 的公式见 式 (1.3) 与 式 (1.4)，其本质是相对熵。
- 评分刻度由基准分数、基准好坏比与 PDO 三者唯一确定，见 式 (1.5) 与 式 (1.6)；PDO 决定曲线的陡峭程度，映射曲线在尾部趋于平坦。
- 预期损失 $EL = PD \times LGD \times EAD$ 是信用风险的第一性公式，风险调整收益 RAROC 把预期损失与经济资本放入分母，见 式 (1.9)。
- 巴塞尔框架下权重法、F-IRB 与 A-IRB 的差别是资本敏感度与治理成本的权衡，内部评级法存在顺周期性。
- 模型监控需要同时跟踪区分度、稳定性与校准度，PSI 见 式 (1.10)，阈值 0.1 与 0.25 分别对应关注与重训。
- 支付基础设施的核心权衡是流动性与结算风险，RTGS 与 DNS 分别偏向两端，ISO 20022 提升了信息载荷但引入映射风险。
- 模型风险治理的关键是独立复现；大语言模型适用于文本密集型辅助工作，不适用于最终数值决策、合规判断与实时执行。
- 另类数据的四项检验为覆盖偏差、幸存者偏差、时点一致性与拥挤度，其中时点错配是回测虚高的最常见原因。

练习与思考

1. 某评分卡的基准设定为 500 分对应好坏比 1 : 1，PDO 为 40。请计算 A 与 B，并求出好坏比为 30 : 1 与 300 : 1 时对应的分数，验证两者之差是否为 2×40 分。
2. 一个特征的五个分箱中，好客户占比依次为 $0.30, 0.25, 0.20, 0.15, 0.10$，坏客户占比依次为 $0.05, 0.10, 0.20, 0.25, 0.40$。请计算各箱 WOE 与整体 IV，并判断该变量的预测力等级。
3. 一笔 500 万元、1 年期的小微贷款，$PD = 4\%$、$LGD = 55\%$、资金成本 3.5%、资本占用比例 8%、资本回报要求 15%、非利息成本 1.0%。请计算使 $RAROC$ 不低于 15% 的最低贷款利率。
4. 某模型的分数分布基准期五箱占比依次为 20%、30%、25%、15%、10%，当期占比依次为 12%、24%、26%、20%、18%。请计算 PSI，指出贡献最大的两箱，并说明是否应触发重训。
5. 请设计一个评估替代数据信用评分公平性的方案：需要说明如何定义受保护群体、选用哪些公平性度量（如不同群体的通过率差异、条件违约率差异）、以及当「公平性」与「区分度」冲突时应如何决策。
6. 某机构打算用大语言模型自动生成贷后催收话术并直接发送给客户。请列出至少四项必须在部署前完成的控制措施，并说明每一项对应哪种风险。
7. 给定一个包含 10 年日频数据的信用卡消费面板，请写出检验其「时点一致性」的具体步骤，并说明如果发现数据曾在第三年被整体修订过，应如何修正回测设计。

延伸阅读

- Siddiqi, N., *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Risk Scorecards*, Wiley, 2006.（分箱、WOE、IV 与评分刻度换算的实务操作手册）
- Basel Committee on Banking Supervision, *Basel III: Finalising Post-crisis Reforms*, Bank for International Settlements, 2017.（信用风险资本计量方法、杠杆率与逆周期缓冲的制度文本）
- Board of Governors of the Federal Reserve System, *SR 11-7: Guidance on Model Risk Management*, 2011.（三道防线与模型验证要求的原始指引，篇幅短但被广泛援引）
- Lopez de Prado, M., *Advances in Financial Machine Learning*, Wiley, 2018.（数据时点错配、特征重要性与回测偏差的系统处理，第 4 至 7 章最相关）
- Provost, F., and Fawcett, T., *Data Science for Business*, O'Reilly, 2013.（把 WOE、IV 这类指标解释为信息增益的通俗读本，适合建立直觉）
- 《中华人民共和国个人信息保护法》与欧盟 *General Data Protection Regulation* 全文。（数据处理的合法性基础、最小必要原则与跨境传输条件的第一手条款）